

SORTING TERMINOLOGY :-

In place Sorting :

→ An in-place sorting algorithm uses constant extra space for producing the output (modifies the given array only). It sorts the list only by modifying the order within the list.

eg. Insertion sort and selection sort are in-place sorting algo as they do not use any additional space for sorting the list.

A typical implementation of Merge sort is not in place.

→ External Sorting : When all data that needs to be sorted cannot be placed in memory at a time, the sorting is called external sorting. Merge sort and its variations are typically used for external sorting.

→ Internal Sorting : When all data is placed in-memory.

Stability in sorting algorithm :

→ A sorting algo is said to be stable, if two elements with equal value (duplicates) appear in the same order in sorted output as they appear in the input array to be sorted. eg. Insertion sort, Bubble sort, Merge sort, Count sort by nature stable.

→ Online algorithm

An online algorithm is one that can process its input piece-by-piece in a serial fashion i.e., in the order that its input is fed to the algorithm, without having the entire input available from the beginning. Whereas Offline is given the whole program data from the beginning and is required to output an answer which solves the problem in hand.

eg: Online : Insertion Sort
Offline : Selection Sort

① Bubble Sort :

→ is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

eg First pass

{ 5, 1, 4, 2, 8 } → { 1 5 4 2 8 }

{ 1 5 4 2 8 } → { 1 4 5 2 8 }

{ 1 4 5 2 8 } → { 1 4 2 5 8 }

{ 1 4 2 5 8 } → { 1 4 2 5 8 }

// since 8 > 5 already in order

→

Second pass

{ 1 4 2 5 8 } → { 1 4 2 5 8 }

{ 1 4 2 5 8 } → { 1 2 4 5 8 }

{ 1 2 4 5 8 } → { 1 2 4 5 8 }

// 5 > 4

{ 1 2 4 5 8 }

→

{ 1 2 4 5 8 }

// 8 > 5

→ Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one whole pass without any swap to know it is sorted.

→

Third pass

— 11 —

— 11 —

— 11 —

— 11 —

Wid bubblesort (int arr[], int n)

{ int i, j;

for (i = 0; i < n-1; i++)

// Last i elements are already in place

Bubbles max

for (j = 0; j < n-i-1; j++)

element to do
right.

if (arr[j] > arr[j+1])

}

swap(&arr[j], &arr[j+1])

② SELECTION SORT:

→ algorithm on array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning.

→ maintains two subarrays in a given array.

① The subarray which is already sorted

② Remaining subarray which is unsorted.

→ In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

eg. 64, 25, 12, 22, 11

→ 11, 25, 12, 22, 64 // find the min element

→ 11, 12, 25, 22, 64 // in arr[0...4] and place it at beginning.

→ 11, 12, 22, 25, 64

→ 11, 12, 22, 25, 64

Wid selectionsort (int arr[], int n)

{ int i, j, min_idx;

for (i = 0; i < n-1; i++)

// find the min element in unsorted array

{ min_idx = i;

for (j = i+1; j < n; j++)

if (arr[j] < arr[min_idx])

min_idx = j;

swap(&arr[min_idx], &arr[i]);

}

}

⑧ Insertion Sort:-

- Simple sorting algorithm that works the way we sort playing cards in our hands.

Algo:-

1. Sort an array of size n .

insertion sort (arr, n)

Loop from $i=1$ to $n-1$

a) Pick element $arr[i]$ and insert it into sorted sequence $arr[0], \dots, i-1$

eg.

4, 3, 2, 10, 12, 1, 5, 6

→

4 3 2, 10, 12, 1, 5, 6

→

3 4 2, 10, 12, 1, 5, 6

→

2 3 4 10, 12, 1, 5, 6

→

2 3 4 10 12, 1, 5, 6

→

2 3 4 10 12 1, 5, 6

→

1 2 3 4 10 12, 5, 6

→

1 2 3 4 5 10 12, 6

→

1 2 3 4 5 6 10 12

→

void insertionSort (int arr[], int n)

{
 int i, key, j;

 for (i = 1; i < n; i++) {

 key = arr[i];

 j = i - 1;

 while (j > 0 & arr[j] > key) {

 arr[j+1] = arr[j];

 j = j - 1;

 }

 arr[j+1] = key;

 }

Heap sort

- Heap sort is a comparison based sorting technique based on Binary Heap data structure.
- Similar to selection sort where we first find the maximum element and place maximum element at the end.
- We repeat the same process for remaining elements.
- We use Binary Heap Data Structure (Complete Binary Tree)



Two types
↓
max heap min heap

Binary heap

A complete binary tree is a binary tree in which every level, except possibly the last is completely filled, and all nodes are as far left as possible.

A Binary heap is a complete binary tree where items are stored in a special order such that value in a parent node is greater (or smaller) than the values in its two children nodes. The former is called as max heap and the latter is called min heap. The heap can be represented by binary tree or array.

→ Why array based representation for Binary heap

Ans: Since a Binary heap is a complete binary tree, it can be easily represented as array and array based representation is space efficient.

If the parent node is stored at index i , the left child can be calculated by $2*i+1$ and right child by $2*i+2$ (assuming indexing starts at 0).

→ Heap sort algorithm for sorting in increasing order.

① Build a max heap from the input data.

② At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of the heap by 1. Finally, heapify the root of tree.

③ Repeat the above steps while size of heap is greater than 1.

→ How to build the heap?

Heapify procedure can be applied to a node only if its children are heapified. So the heapification must be performed in the bottom up order.

→ Build heap sort (int array)

```
1. Build Heap (reverse array)
for (int i = n/2 - 1; i >= 0; i--)
    heapify(arr, n, i);
for (int i = n - 1; i > 0; i--)
    if (more current root to end)
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
```

Mergesort:

→ Like Quicksort, Mergesort is a Divide and Conquer algorithm. ① It divides input array in two halves, ② calls itself for the two halves and then merges the two sorted halves.

→ The merge (arr, l, m, r) is key process that assumes that $arr[l, \dots, m]$ and $arr[m+1, r]$ are sorted and merges the two sorted sub-arrays into one.

→ Mergesort($arr[]$, l, r)

if $r > l$

1. Find the middle point to divide the array into two halves:

$$\text{middle } m = (l+r)/2$$

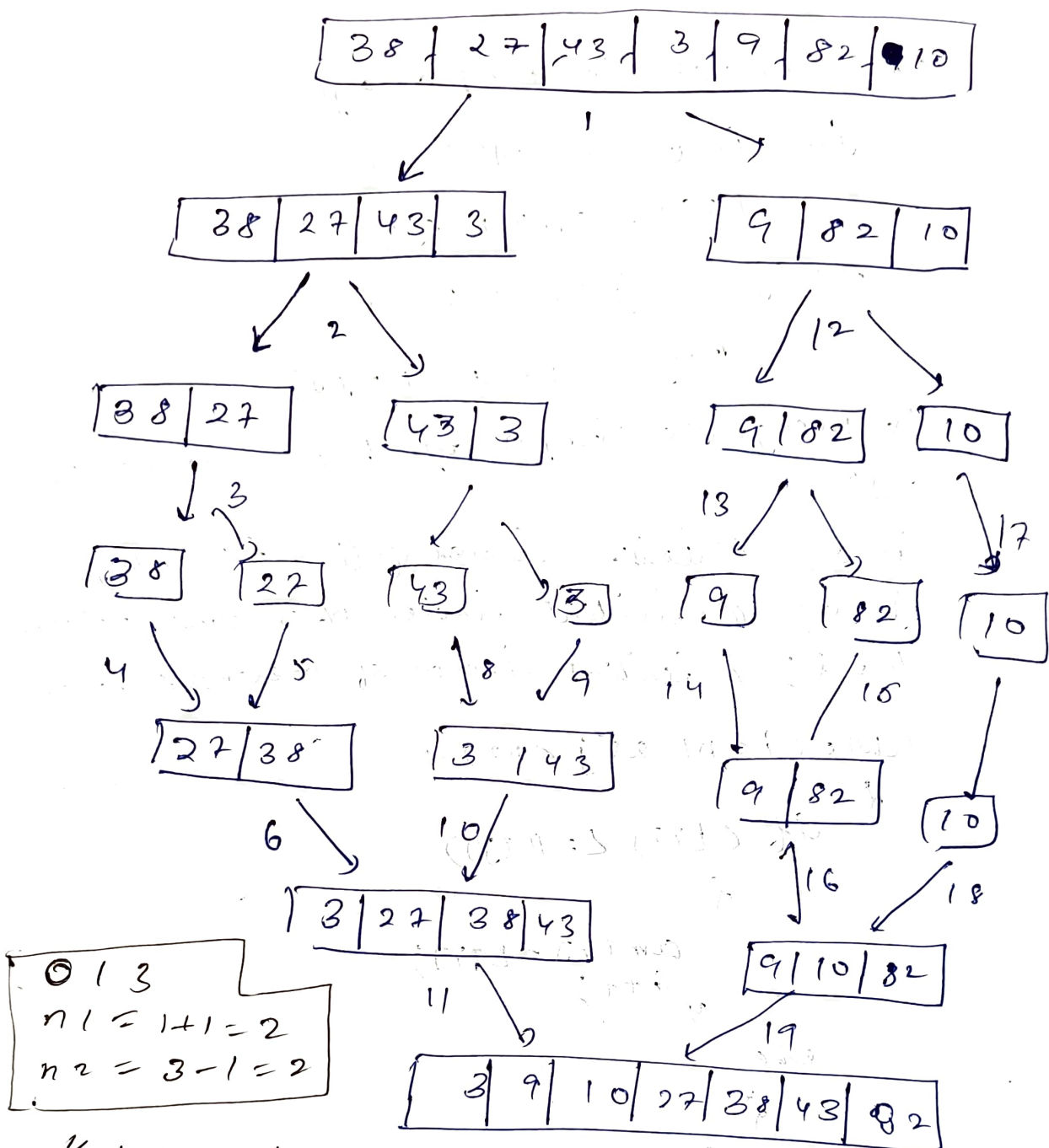
2. call mergesort for first half:
call mergesort(arr, l, m)

3. call mergesort for second half:
call mergesort($arr, m+1, r$)

4. Merge the two halves sorted in step 2 and 3:

call merge(arr, l, m, r)

→ see diagram next page.



// merge two sub arrays of arr[].

// First subarray is $arr[l, \dots, m]$ & second is $arr[m+1, \dots, r]$.

```
void merge (int arr[], int l, int m, int r)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    // Create two temp arrays
```

```
    int L[n1], R[n2];
```

/* Copy data to temp arrays

for ($i=0$; $i < n1$; $i++$)

$L[i] = arr1[i];$

for ($j=0$; $j < n2$; $j++$)

$R[j] = arr[m+1+j];$

/* Merge the temp arrays back into arr1...

$i=0$; // initial index of first subarray

$j=0$; // initial index of second subarray

$k=1$; // initial index of merged subarray

while ($i < n1$ & $j < n2$)

if ($L[i] <= R[j]$)

$arr[k] = L[i];$
 $i++;$

else

$arr[k] = R[j];$

$j++;$

$k++;$

/* Copy the remaining elements of L[], if there are any */

while ($i < n1$)

{ $arr[k] = L[i];$

$i++;$

$k++;$

}

1. Copy the remaining element of RTJ if there are any.

while ($j < h$)

{

arr[k] = $RTJ[j]$;

$j++$;

$k++$;

}

}

* l is for left index and r is right index of the subarray of arr to be sorted.

void mergesort (int arr[], int l, int r)

{

if ($l < r$)

same as $(l+r)/2$, but avoids overflow for large l and h .

int $m = l + (r-l)/2$;

// sort first and second halves

mergesort (arr, l, m);

mergesort (arr, m+1, r);

merge (arr, l, m, r);

Q Mergesort and Quicksort are both divide and Conquer algorithms.

Mergesort: bottom up algorithm

Quicksort: Top down fashion. (First element always will be in correct position.)

Quicksort:

- Best and Average sorting complexity = $O(n \log n)$
- Worst sorting complexity = $O(n^2)$
- pivot element will be in its correct position.
- Like mergesort, Quicksort is a divide and conquer algorithm.
- It picks ^{an} element as pivot and partitions the given array around the picked pivot.
- many different versions of Quicksort are there. They pick pivot element in different ways.
 - ① Always pick first element as pivot.
 - ② Always pick last element as pivot.
 - ③ Pick a random element as pivot.
 - ④ Pick median as pivot.
- The key process in Quicksort is partition().
- Target of partition is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (than x) before x , and put all greater elements (greater than x). All should be done in linear time.

void swap(int *a, int *b)

{

int t = *a;

*a = *b;

*b = t;

}

① int partition (int a[], int low, int high)

{
 int pivot, index, i;
 index = low;
 pivot = high;

for (i = low; i < high; i++)

{ if (a[i] < a[pivot])

[7|8|6|3|1|9]

{

Swap (&a[i], &a[index]);

index++;

Swap (&a[pivot], &a[index]);

return index;

② int quicksort (int a[], int low, int high)

{
 int pivot, index;
 if (low < high)

{ pivot = RandomPivotPartition (a, low,

quicksort (a, low, pivot-1);

quicksort (a, pivot+1, high);

return 0;

int randomPivotPartition (int a[],
 int low, int high)

{ int prv, n, temp;

n = rand();

prv = low + n % (high - low + 1);

Swap (&a[prv], &a[high]);

return partition (a, low, high);

Analysis of the Quicksort

→ Time taken by Quicksort in general can be written as following

$$T(n) = T(k) + T(n-k-1) + \Theta(n)$$

↓
for the partition process

The first two terms are for two recursive calls, k is the no. of elements which are smaller than pivot.

→ The time taken by Quicksort depends upon the input array and partition strategy.

1. Worst case occurs when the partition process always picks the greatest (or smallest) element as pivot. So, the worst case will occur when the array is already sorted in increasing or decreasing order.

So, now, $T(n) = T(n-1) + O(n)$ ∵ $k=0$

$$(n-1) \left\{ \begin{array}{l} T(n-2) \\ T(n-3) \\ T(n-4) \\ \vdots \\ T(1) \end{array} \right.$$

$$O(n^2) = O(n-1) + O(n)$$

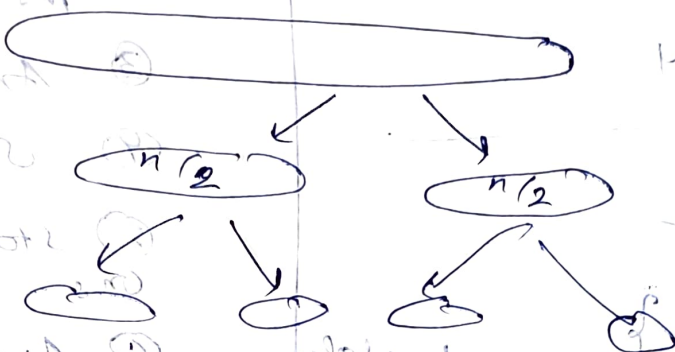
2. Best case occurs when the partition process always picks the middle element as pivot.

$$T(n) = 2T(n/2) + \Theta(n)$$

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$



$$2^k = n$$

Now,

$$k \log_2 2 = \log_2 n$$

$$k = \log_2 n$$

(In k steps $\log_2 n$ complexity)

$$O(n \log n)$$

3. Average case

Here we need to consider all possible permutations of array and calculate time taken by every permutation which doesn't look easy.

We can get an idea of average case by considering the case where partition puts $O(n/9)$ elements in one set and $O(9n/10)$ in other set.

$$T(n) = T(n/9) + T(9n/10) + \Theta(n)$$

Worst time complexity is $O(n^2)$

→ Worst case rarely occurs for a given type of data. However, mergesort is generally considered better when data is huge and stored in external storage.